

APL 745

Optimization - II

Dr. Rajdip Nayek

Block V, Room 418D

Department of Applied Mechanics

Indian Institute of Technology Delhi

E-mail: rajdipn@am.iitd.ac.in

Course website: <https://www.cscdm.in/courses/deep-learning-for-mechanics>

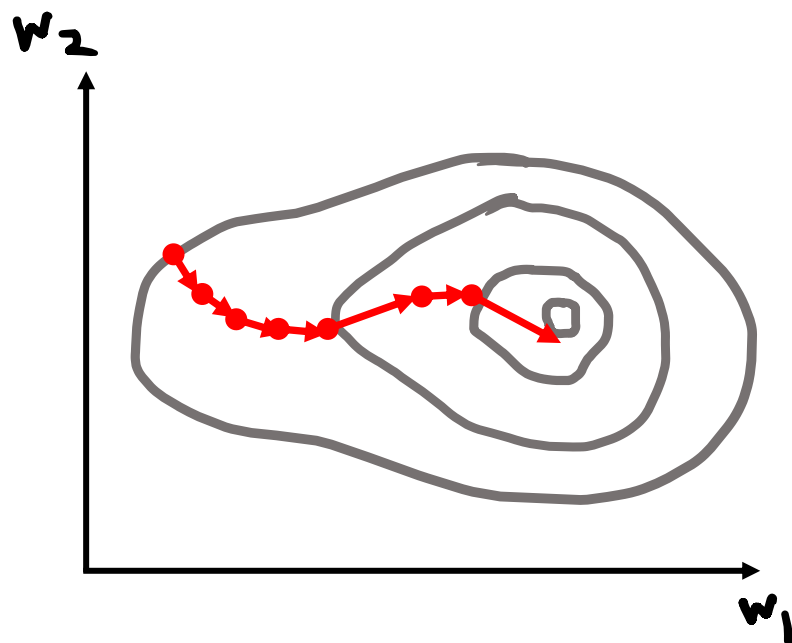
Overview

- Last class, we saw
 - Gradient Descent (uses full training data to get gradients)
 - Stochastic Gradient Descent (uses individual training data to get gradients)
 - Mini-batch Gradient Descent (uses a batch of training data to get gradients)
- Training neural networks is non-convex, most often you will be converging to a local minimum
- A good initialization of the weights and biases will prevent symmetry and issues related to saturated and dead units
- **Ill-conditioning** is a big problem
 - It causes **slow learning** and **oscillations**
 - Normalizing the input helps to reduce the problem
 - Batch-normalizing the activations of the hidden layers also helps
 - However, even with normalization, ill-conditioning is not completely removed

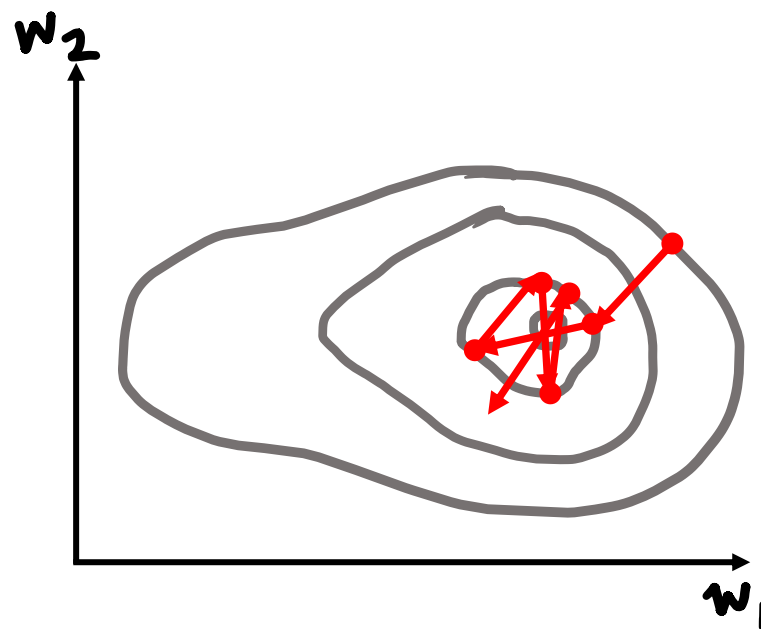
Ill-conditioning related problems

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \nabla \mathcal{L}(\theta^{(t)})$$

- **Gentle slope** $\rightarrow$ magnitude of gradient is smaller $\rightarrow$ GD takes small steps
- **Steep slope** $\rightarrow$ magnitude of gradient is large $\rightarrow$ GD takes big steps



Slow learning



Oscillations

Note: Oscillations are different than fluctuations in SGD

Overview

- This lecture will be on different modifications that can be applied to GD, SGD or mini-batch GD to improve convergence to a solution (possibly a local minimum)
- Two lines of improvements to traditional GD (or SGD or mini-batch GD)

$$\theta^{(t+1)} \leftarrow \theta^{(t)} + \eta \nabla_{\theta} \mathcal{L}(\theta^{(t)})$$

learning rate

gradient of loss w.r.t. parameter $\theta^{(t)}$

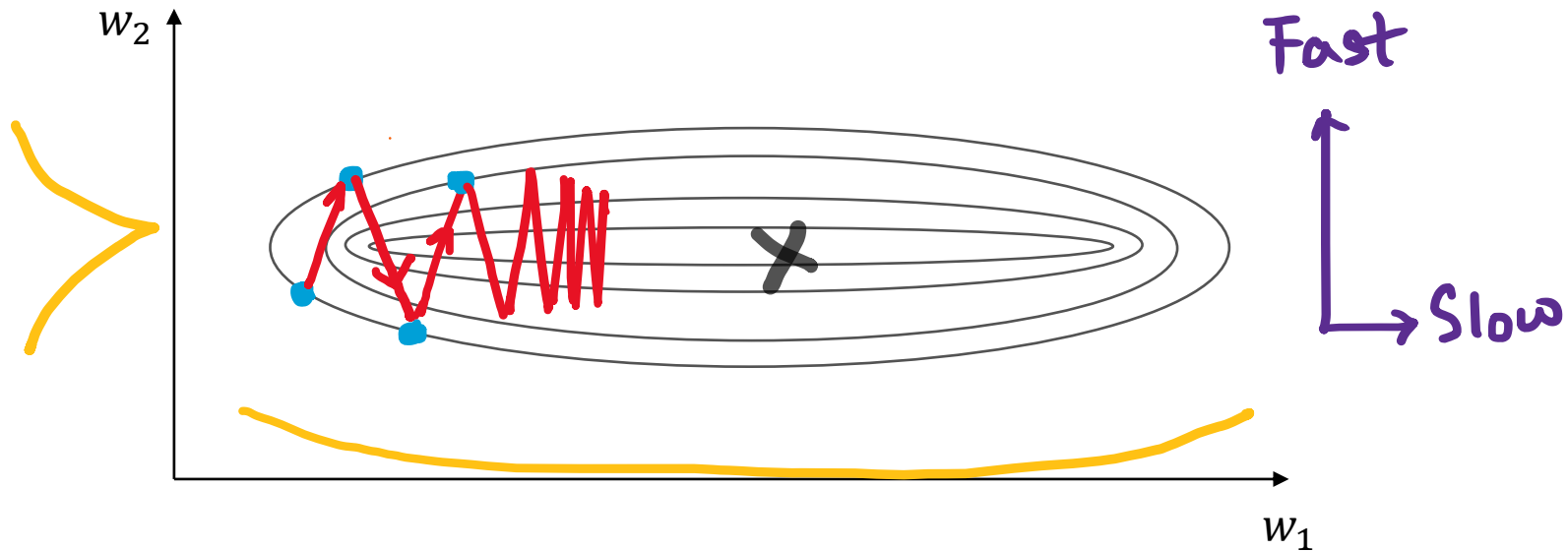
Adaptively modify the gradients to accelerate learning	Adaptively modify the learning rate to prevent end oscillations
• Momentum-based gradients • Nesterov accelerated gradients	• AdaGrad • RMSProp

ADAM

- We will demonstrate these modifications on GD, but they are equally applicable to SGD and mini-batch GD as well

Slow learning and Oscillations

- GD takes faster jumps on steeper slopes and slower steps on gentler slopes



- But we want to take smaller steps in the steeper direction and larger steps in the flatter direction

Want to
move
slower

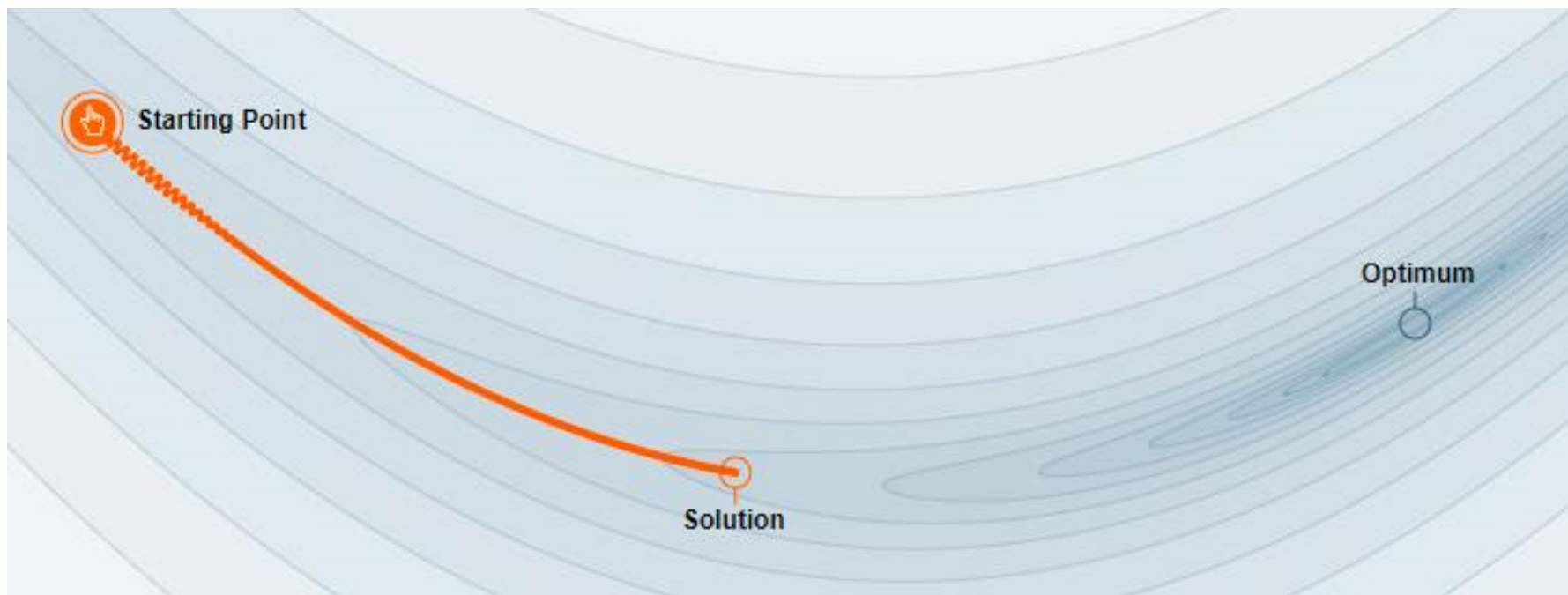
w_2



w_1

Want to move faster

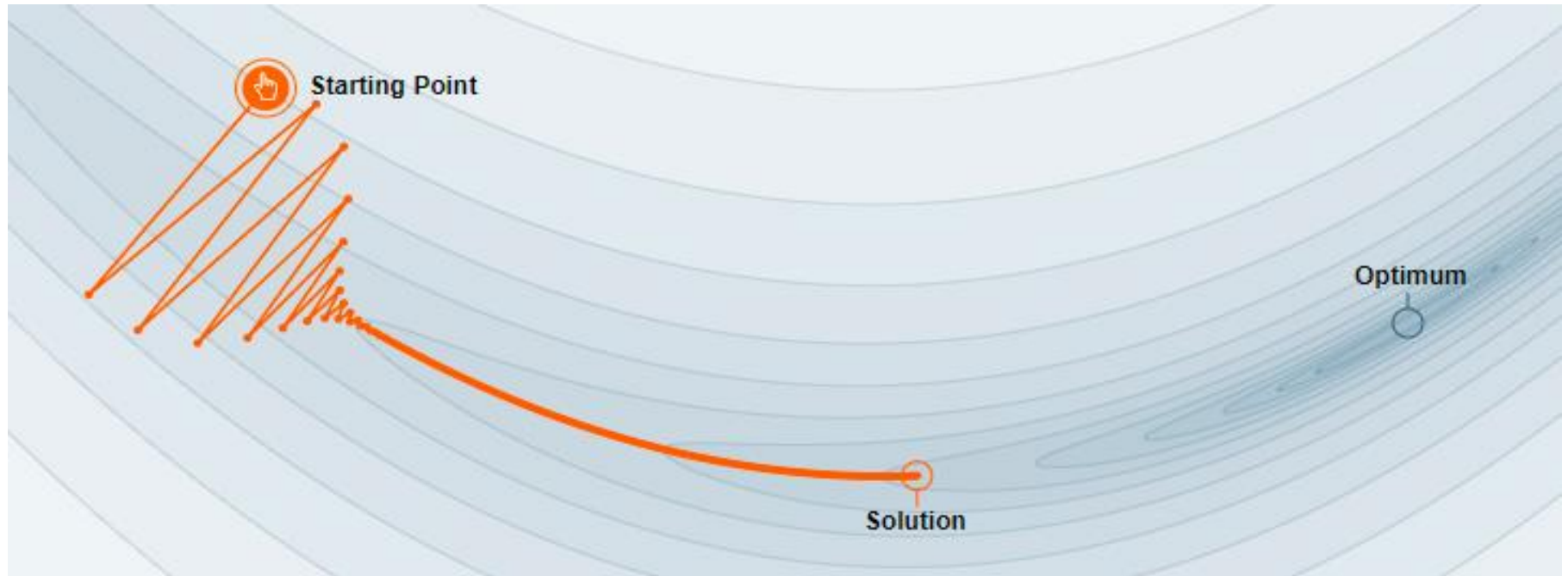
Slow learning on gentler slopes



<https://distill.pub/2017/momentum/>

Slow learning along gentle slopes, many steps taken to converge

Oscillations across steeper slopes



<https://distill.pub/2017/momentum/>

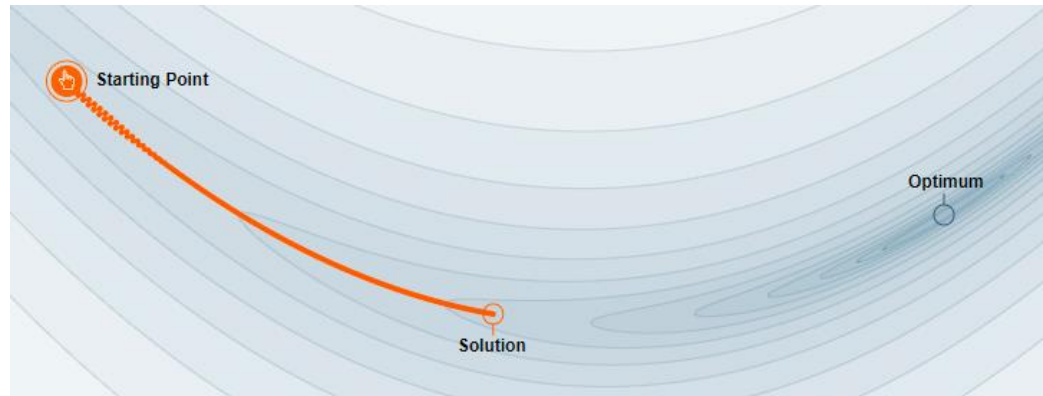
Oscillations across steep slopes

Can we accelerate learning by looking at the past behaviour? Yes, use momentum

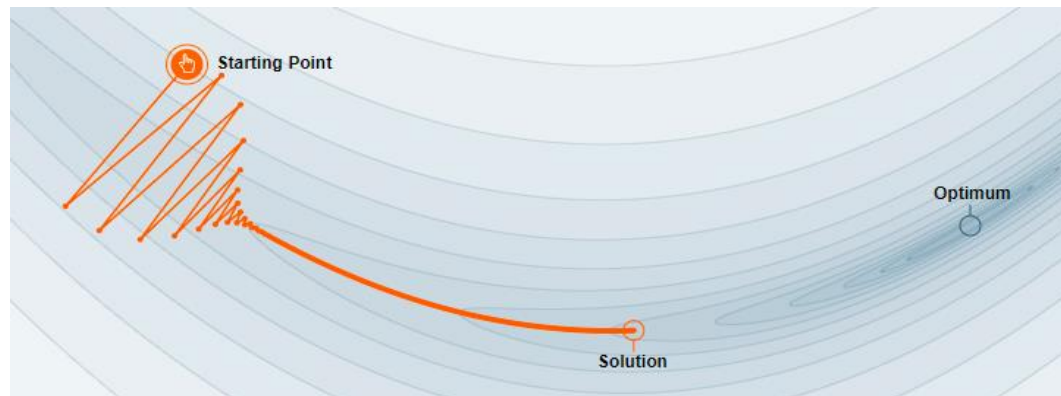
Momentum-based Gradient Descent

Intuition

- If you are repeatedly being asked to move in the same direction, then you should gain some confidence and start taking bigger steps in that direction



- If you move back-and-forth in different directions (i.e oscillations), then you should take smaller steps in the oscillatory directions



Momentum-based Gradient Descent

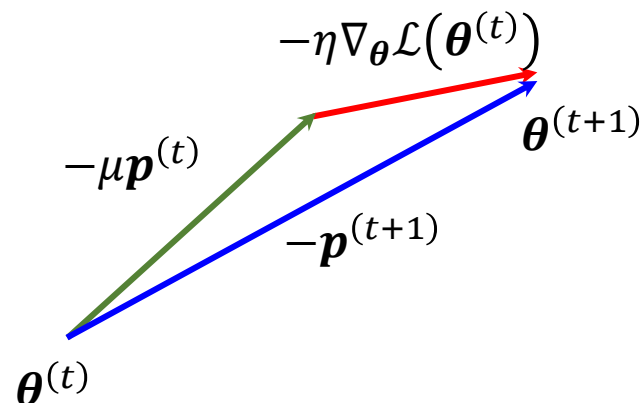
Intuition

- If you are repeatedly being asked to move in the same direction, then you should gain some confidence and start taking bigger steps in that direction
- If you move back-and-forth in different directions (i.e oscillations), then you should take smaller steps in the oscillatory directions
- **Gain momentum** by looking at the **past history of past gradients**

Update rule

- Compute momentum
$$\mathbf{p}^{(t+1)} \leftarrow \mu \mathbf{p}^{(t)} + \eta \nabla_{\theta} \mathcal{L}(\theta^{(t)})$$
- Perform parameter update
$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \mathbf{p}^{(t+1)}$$

history
vector



Momentum-based Gradient Descent

Update rule

- Compute momentum

$$\mathbf{p}^{(t+1)} \leftarrow \mu \mathbf{p}^{(t)} + \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)})$$

- Perform parameter update

$$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} - \mathbf{p}^{(t+1)}$$

- η is the learning rate, just like in gradient descent
- μ is a damping parameter, and should satisfy $0 \leq \mu \leq 1$
- If $\mu = 0$, one recovers traditional gradient descent
- μ should be slightly less than 1 (e.g. 0.9 or 0.99)

Momentum-based Gradient Descent

Update rule

- Compute momentum

$$\mathbf{p}^{(t+1)} \leftarrow \mu \mathbf{p}^{(t)} + \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)})$$

- Perform parameter update

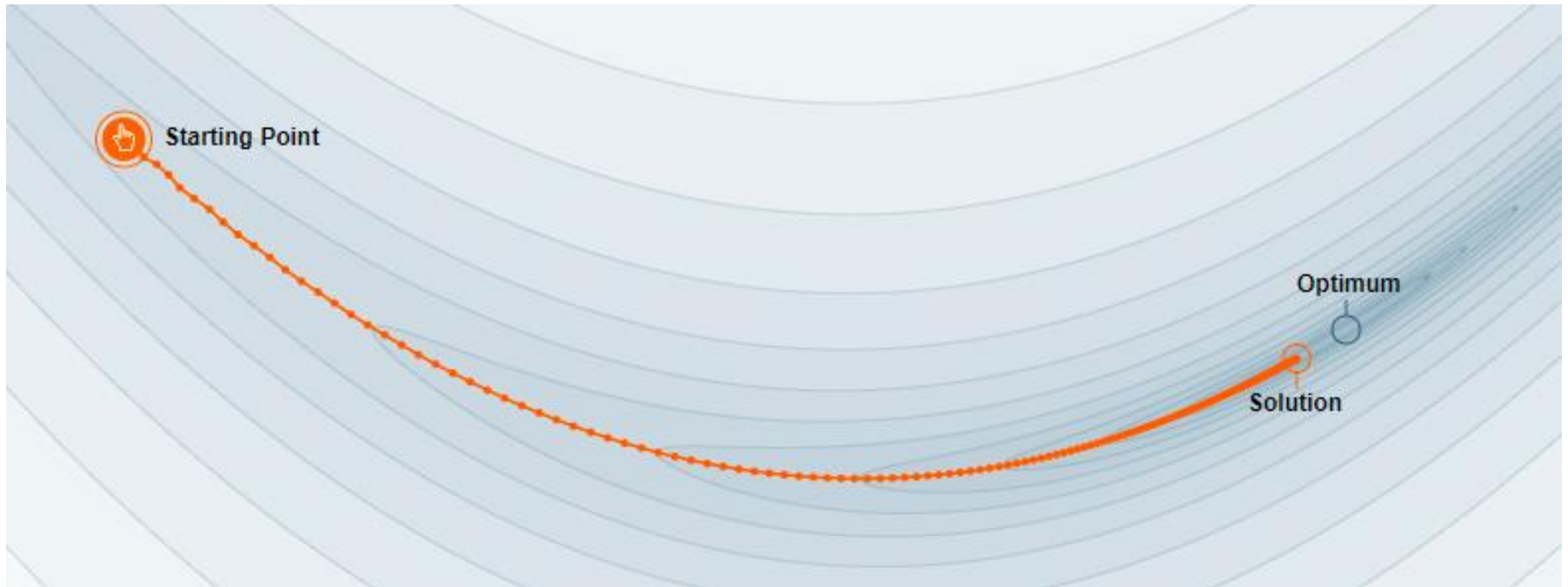
$$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} - \mathbf{p}^{(t+1)}$$

```
theta, eta, epochs = initial_th, 0.1, 1000
prev_p, mu, N = 0, 0.9, 10000
for i in range(epochs):
    dtheta = 0 # initialize increment to zero
    for x,t in zip(X,T):
        dtheta += grad_theta(theta, x, t)

    # Momentum update
    new_p = mu * prev_p + eta * dtheta/N
    # Parameter update
    theta = theta - new_p

    prev_p = new_p
```

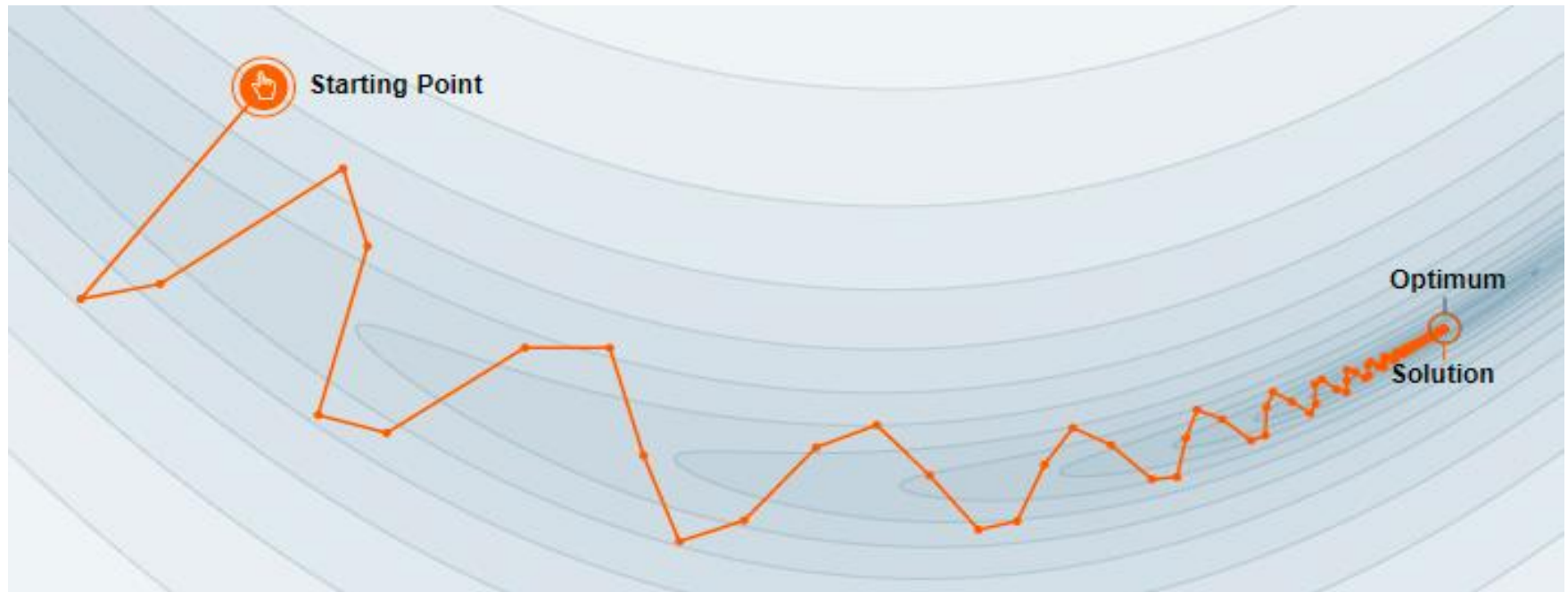
Working of momentum-based GD



<https://distill.pub/2017/momentum/>

Moves faster on gentler slopes

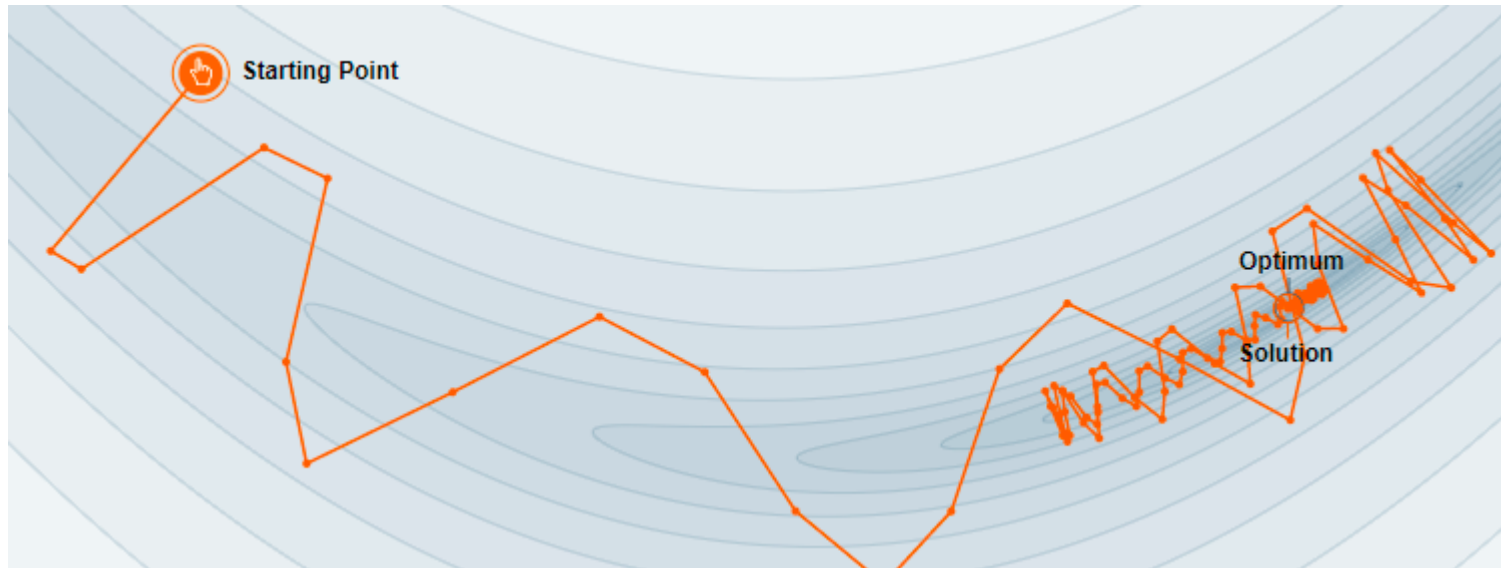
Working of momentum-based GD



<https://distill.pub/2017/momentum/>

Reduces oscillations with iterations on steeper slopes

Observations on momentum-based GD



- **Is moving fast always good?** A problem with momentum is that acceleration can sometimes cause the search to overshoot the minima
- Momentum based gradient descent can oscillate in and out of the optimum (minimum) due to the momentum
- Despite these oscillations it still converges faster than traditional Gradient Descent
- **Can we do something to prevent these oscillations due to momentum?**
- Yes, let's look at Nesterov accelerated gradient (NAG)

Nesterov accelerated gradient (NAG)

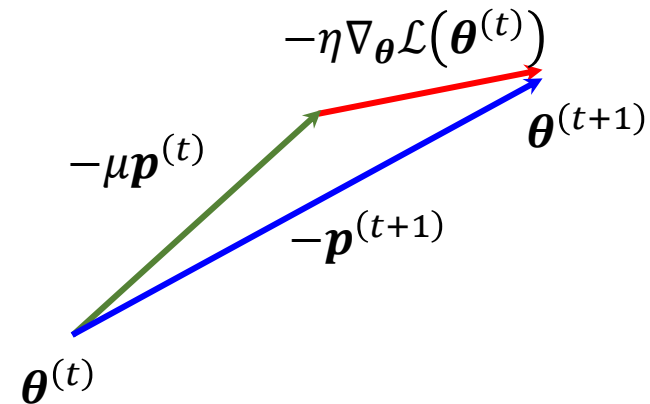
- Recall momentum uses gradients of past and gradient computed at $\theta^{(t)}$

- Update the momentum

$$\mathbf{p}^{(t+1)} \leftarrow \underbrace{\mu \mathbf{p}^{(t)}}_{\text{past}} + \underbrace{\eta \nabla_{\theta} \mathcal{L}(\theta^{(t)})}_{\text{current}}$$

- Perform parameter update

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \mathbf{p}^{(t+1)}$$



- NAG:** Look into the future to see how much momentum is required

- Look ahead point

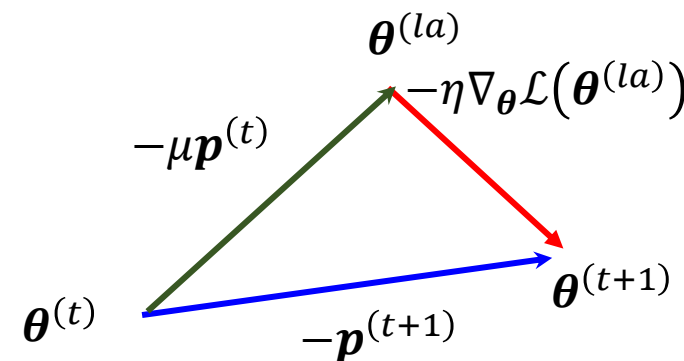
$$\theta^{(la)} \leftarrow \theta^{(t)} - \mu \mathbf{p}^{(t)}$$

- Use look-ahead point to get current gradient

$$\mathbf{p}^{(t+1)} \leftarrow \mu \mathbf{p}^{(t)} + \eta \nabla_{\theta} \mathcal{L}(\theta^{(la)})$$

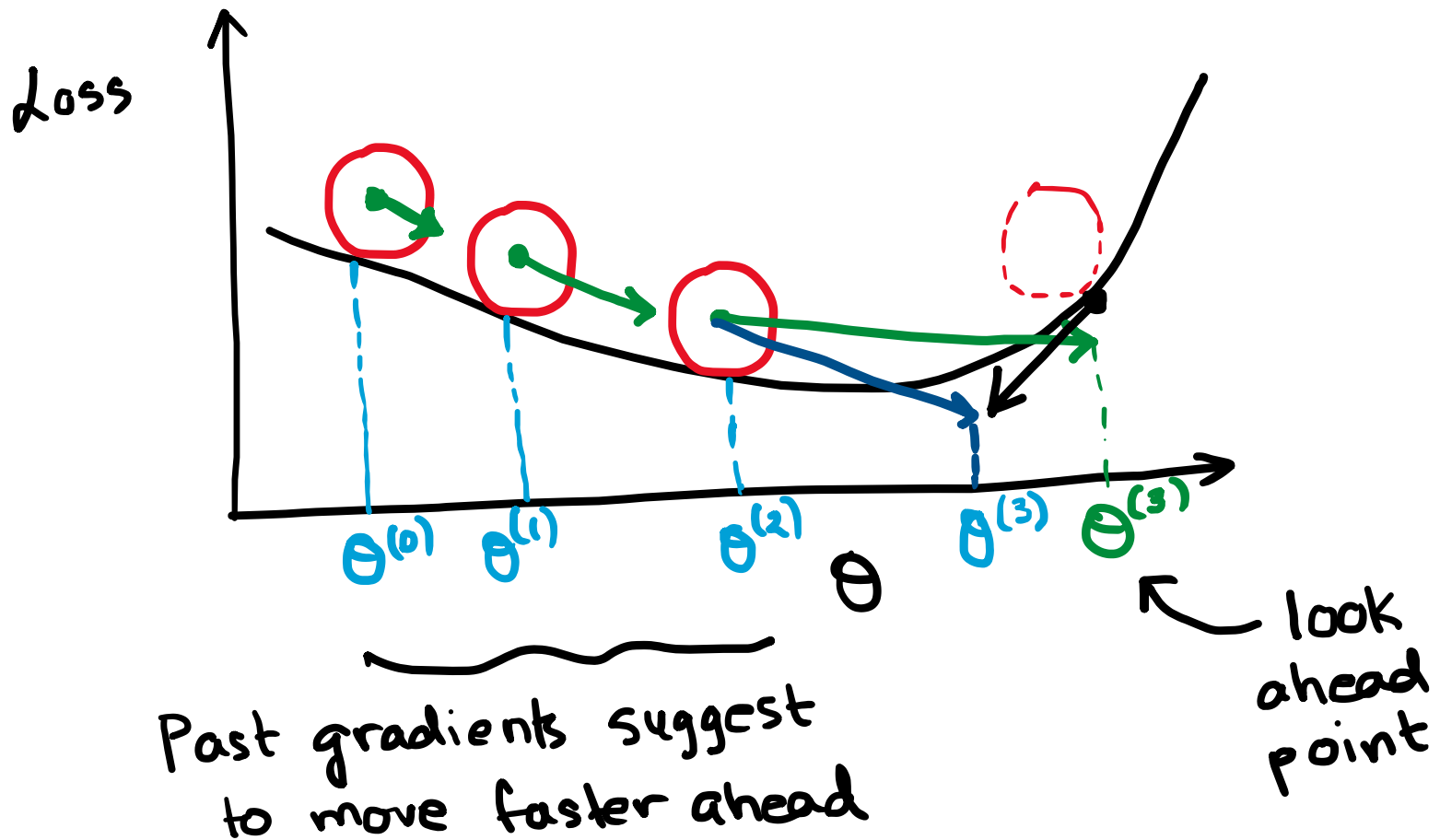
- Perform parameter update

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \mathbf{p}^{(t+1)}$$



Nesterov accelerated gradient (NAG)

- **First** make a big jump in the direction of the previous accumulated gradient
- **Then** measure the gradient where you end up and make a correction



Nesterov accelerated gradient (NAG)

```
theta, eta, epochs = initial_th, 0.1, 1000
prev_p, mu, N = 0, 0.9, 10000
for i in range(epochs):
    dtheta = 0 # initialize increment to zero
    # Look ahead point
    theta_future = theta - mu * prev_p

    for x,t in zip(X,T):
        # Compute gradient at future point
        dtheta += grad_theta(theta_future, x, t)

    # Momentum update
    new_p = mu * prev_p + eta*dtheta/N
    # Parameter update
    theta = theta - new_p

    prev_p = new_p
```

*Dividing
by N
training
examples
to calculate
gradient

Adaptive learning rates

$$\theta^{(t+1)} \leftarrow \theta^{(t)} + \eta \nabla_{\theta} \mathcal{L}(\theta^{(t)})$$

learning
rate

gradient of loss
w.r.t. parameter $\theta^{(t)}$

Adaptively **modify the gradients**
to accelerate learning

- Momentum-based gradients
- Nesterov accelerated gradients

Adaptively **reduce the learning rate**
to prevent end oscillations

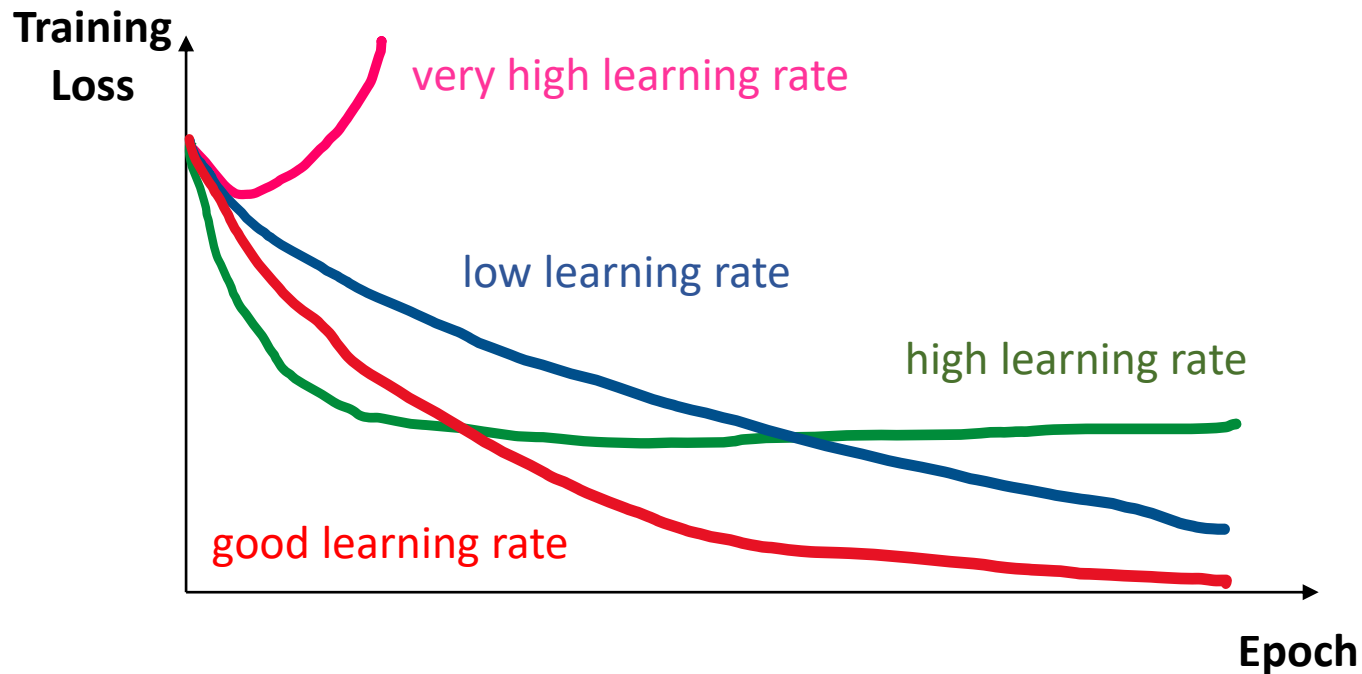
- AdaGrad
- RMSProp

ADAM

Learning rate

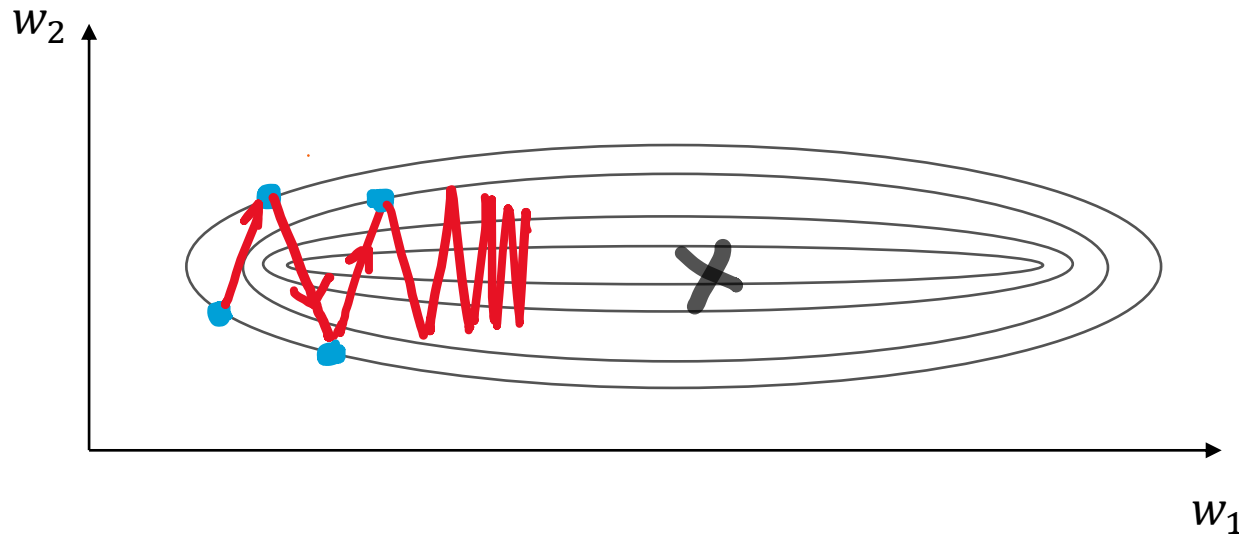
- Ideal learning rate η should be
 - Not too big (loss function may blow up, oscillations around minima)
 - Not too small (takes longer to converge)

$$\theta^{(t+1)} \leftarrow \theta^{(t)} + \eta \nabla_{\theta} \mathcal{L}(\theta^{(t)})$$



Learning rate

- One learning rate for all parameters is not good
- Can we tune the learning rate for each parameter directions separately?
 - E.g. We may want to move fast in one parameter direction compared to other
- Consider this toy problem with two parameters, we want to
 - Aggressively reduce learning rate in vertical direction
 - Gradually reduce learning rate in horizontal direction



Idea: Decay the learning rate for parameters in proportion to their **gradient magnitude history**

Adaptive Gradients (AdaGrad)

- AdaGrad uses the magnitude of the gradient as a means of adjusting how quickly learning should occur - coordinates with large gradient magnitudes are provided with a smaller learning rate

Update rule for AdaGrad

- Get gradient

$$\mathbf{g}^{(t)} = \nabla_{\theta} \mathcal{L}(\theta^{(t)})$$

- Accumulate past gradient magnitudes in a history vector

history vector $\longrightarrow$ $\mathbf{s}^{(t+1)} \leftarrow \mathbf{s}^{(t)} + \underbrace{(\mathbf{g}^{(t)})^2}_{\text{magnitude of gradient}}$

- Perform parameter update

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \frac{\eta}{\sqrt{\mathbf{s}^{(t+1)} + \epsilon}} \mathbf{g}^{(t)}$$

elementwise
squaring

- Here, the squaring and update operation is applied elementwise

Adaptive Gradients (AdaGrad)

Update rule for AdaGrad

- Get gradient

$$\mathbf{g}^{(t)} = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)})$$

- Accumulate past gradient magnitudes in a history vector

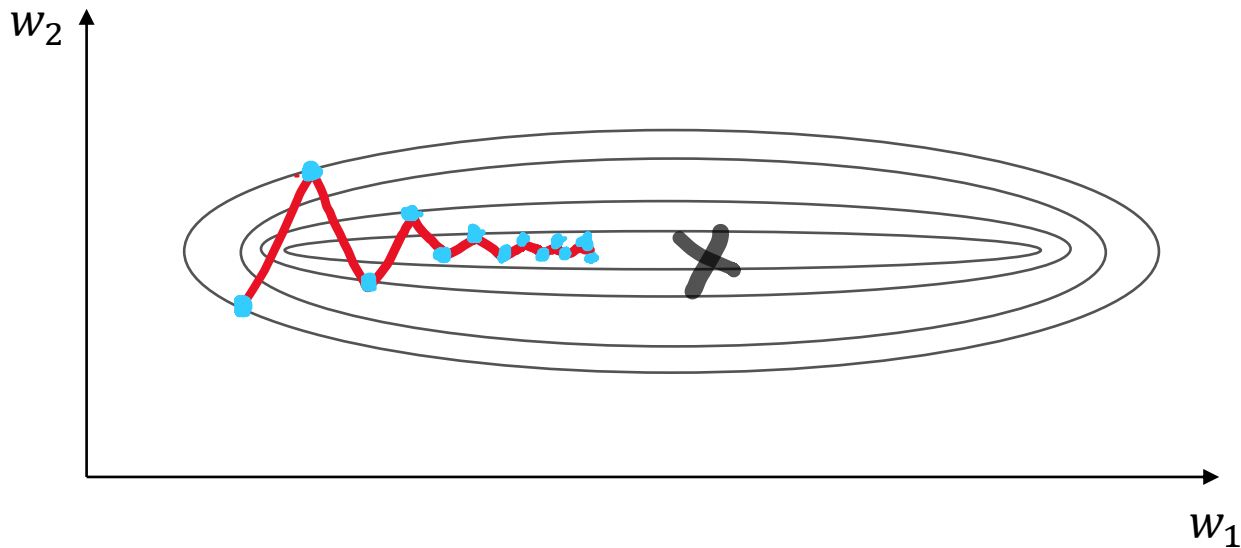
history vector $\longrightarrow \mathbf{s}^{(t+1)} \leftarrow \mathbf{s}^{(t)} + (\mathbf{g}^{(t)})^2$

- Perform parameter update

$$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} - \frac{\eta}{\sqrt{\mathbf{s}^{(t+1)} + \epsilon}} \mathbf{g}^{(t)}$$

- η is the learning rate and ϵ is a small additive constant (10^{-8}) that ensures that we do not divide by 0
- The squaring operation gets rid of signs (directions) of the gradients accumulated, hence we keep the magnitudes of gradients

AdaGrad



- However, Adagrad decays the learning rate very aggressively (since it accumulates all past gradient magnitudes and the denominator grows)
- As a result, during later epochs, some of the parameters will start receiving very small updates because of the decayed learning rate
- **How can we prevent rapid growth of the denominator?**
- Let's look at RMSProp

Root Mean Square Propagation (RMSProp)

Update rule for RMSProp **Trick:** Focus more on the **recent** past

- Get gradient

$$\mathbf{g}^{(t)} = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)})$$

- Accumulate moving average over the history vector

history vector $\longrightarrow$ $\mathbf{s}^{(t+1)} \leftarrow \beta \mathbf{s}^{(t)} + (1 - \beta)(\mathbf{g}^{(t)})^2$

- Perform parameter update

$$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} - \frac{\eta}{\sqrt{\mathbf{s}^{(t+1)} + \epsilon}} \mathbf{g}^{(t)}$$

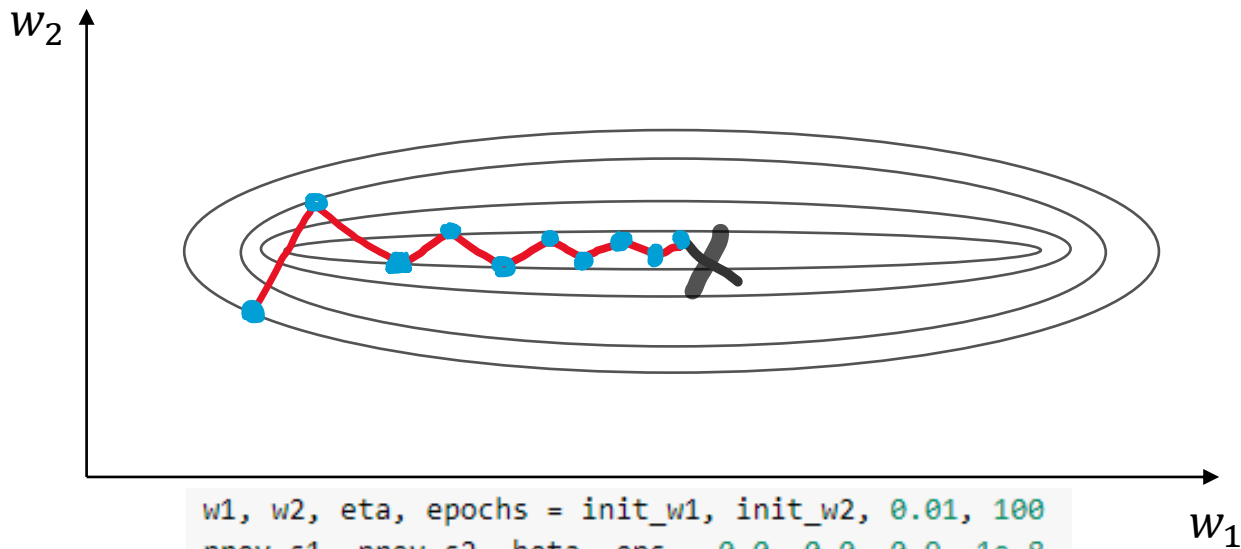
$$0 \leq \beta \leq 1$$

$$\beta = 0.85, 0.9, 0.95$$

- β determines how much preference to give to the past gradient magnitudes
- RMSProp uses **exponential moving averages** of the past gradient magnitudes

$$\mathbf{s}^{(t+1)} = (1 - \beta) \left[(\mathbf{g}^{(t)})^2 + \beta (\mathbf{g}^{(t-1)})^2 + \beta^2 (\mathbf{g}^{(t-2)})^2 + \dots \right]$$

RMSProp



```
w1, w2, eta, epochs = init_w1, init_w2, 0.01, 100
prev_s1, prev_s2, beta, eps = 0.0, 0.0, 0.9, 1e-8
```

$N=10000$

```
for i in range(max_epochs):
    dw1, dw2 = 0.0, 0.0    # initialize increment
```

```
    for x,t in zip(X,T):
```

```
        dw1 += grad_w1(w1, w2, x, t)
```

```
        dw2 += grad_w2(w1, w2, x, t)
```

$dw1 = dw1 / N$ $dw2 = dw2 / N$

```
    # Gradient magnitude history vector
```

```
    new_s1 = beta * prev_s1 + (1-beta) * dw1**2
```

```
    new_s2 = beta * prev_s2 + (1-beta) * dw2**2
```

```
    # Parameter update
```

```
    w1 = w1 - (eta / np.sqrt(new_s1 + eps)) * dw1
```

```
    w2 = w2 - (eta / np.sqrt(new_s2 + eps)) * dw2
```

Adaptive Moment Estimation (ADAM)

Idea

- Do everything that RMSProp does to solve the decay problem of Adagrad
- Plus use momentum based on a cumulative history of the gradients
- ADAM = RMSProp + Momentum

- Get gradient

$$\mathbf{g}^{(t)} = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)})$$

- Compute momentum

$$\mathbf{p}^{(t+1)} \leftarrow \beta_1 \mathbf{p}^{(t)} + (1 - \beta_1) \mathbf{g}^{(t)}$$

- Accumulate past gradient step sizes in a history vector

$$\mathbf{s}^{(t+1)} \leftarrow \beta_2 \mathbf{s}^{(t)} + (1 - \beta_2) (\mathbf{g}^{(t)})^2$$

- Perform parameter update

$$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} - \frac{\eta}{\sqrt{\mathbf{s}^{(t+1)} + \epsilon}} \mathbf{p}^{(t+1)}$$

In practice

$$\beta_1 = 0.9$$

$$\beta_2 = 0.999$$

ADAM with bias correction

- Note that if we initialize $\mathbf{p}^{(0)} = \mathbf{s}^{(0)} = \mathbf{0}$, we have a significant amount of bias initially leading to very small updates to $\boldsymbol{\theta}$
- Bias corrections are introduced to renormalize terms

- Get gradient

$$\mathbf{g}^{(t)} = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)})$$

- Compute momentum and correct for bias

$$\mathbf{p}^{(t+1)} \leftarrow \beta_1 \mathbf{p}^{(t)} + (1 - \beta_1) \mathbf{g}^{(t)},$$

$$\hat{\mathbf{p}}^{(t+1)} = \frac{\mathbf{p}^{(t+1)}}{1 - \beta_1^{t+1}}$$

Bias
corrected
terms

- Accumulate past gradient step sizes and correct for bias

$$\mathbf{s}^{(t+1)} \leftarrow \beta_2 \mathbf{s}^{(t)} + (1 - \beta_2) (\mathbf{g}^{(t)})^2,$$

$$\hat{\mathbf{s}}^{(t+1)} = \frac{\mathbf{s}^{(t+1)}}{1 - \beta_2^{t+1}}$$

- Perform parameter update

$$\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} - \frac{\eta}{\sqrt{\hat{\mathbf{s}}^{(t+1)} + \epsilon}} \hat{\mathbf{p}}^{(t+1)}$$

Bias correction for exponential moving average

- Note that if we initialize $\mathbf{p}^{(0)} = \mathbf{s}^{(0)} = \mathbf{0}$, we have a significant amount of bias initially leading to very small updates to θ
- To avoid bias, bias corrections are introduced to renormalize terms

$$p^{(t+1)} = \beta_1 p^{(t)} + (1 - \beta_1) g^{(t)}$$

$$\begin{aligned} p^{(1)} &= 0.98 p^{(0)} + 0.02 g^{(0)} \\ &= 0.98(0) + 0.02 g^{(0)} \\ &= \underline{0.02 g^{(0)}} \quad \text{small} \end{aligned}$$

$$\begin{aligned} p^{(2)} &= 0.98 p^{(1)} + 0.02 g^{(1)} \\ &= 0.98(0.02 g^{(0)}) + 0.02 g^{(1)} \\ &= \underline{0.0196 g^{(0)} + 0.02 g^{(1)}} \quad \text{small} \end{aligned}$$

$$\text{Say, } \beta_1 = 0.98$$

$$\begin{aligned} \hat{p}^{(1)} &= \frac{0.02 g^{(0)}}{1 - (0.98)^1} \\ &= g^{(0)} \end{aligned}$$

$$\hat{p}^{(2)} = \frac{0.0196 g^{(0)} + 0.02 g^{(1)}}{1 - (0.98)^2}$$

ADAM with bias correction

```
w1, w2, eta, epochs = init_w1, init_w2, 0.01, 100
prev_s1, prev_s2 = 0.0, 0.0
prev_p1, prev_p2 = 0.0, 0.0
beta1, beta2, eps = 0.9, 0.999, 1e-8
```

$N = 10000$ # training examples

```
for i in range(max_epochs):
```

```
    dw1, dw2 = 0.0, 0.0    # initialize increment
```

```
    for x, t in zip(X, T):
```

```
        dw1 += grad_w1(w1, w2, x, t)
```

```
        dw2 += grad_w2(w1, w2, x, t)
```

$dw1 = dw1 / N$ $dw2 = dw2 / N$

```
    new_p1 = beta1 * prev_p1 + (1 - beta1) * dw1
```

```
    new_p2 = beta1 * prev_p2 + (1 - beta1) * dw2
```

Momentum {

Bias corr. {

```
    new_p1_bar = new_p1 / (1 - math.pow(beta1, i+1))
```

```
    new_p2_bar = new_p2 / (1 - math.pow(beta1, i+1))
```

Grad mag. {

```
    new_s1 = beta2 * prev_s1 + (1 - beta2) * dw1**2
```

```
    new_s2 = beta2 * prev_s2 + (1 - beta2) * dw2**2
```

Bias corr. {

```
    new_s1_bar = new_s1 / (1 - math.pow(beta2, i+1))
```

```
    new_s2_bar = new_s2 / (1 - math.pow(beta2, i+1))
```

Final update {

```
    w1 = w1 - (eta / np.sqrt(new_s1_bar + eps)) * new_p1_bar
```

```
    w2 = w2 - (eta / np.sqrt(new_s2_bar + eps)) * new_p2_bar
```

Okay so what to use in practice?

Which algorithm to use in practice?

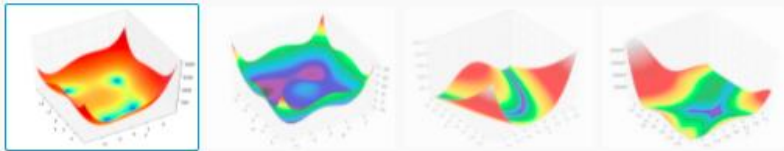
- ADAM seems to be more or less the default choice now ($\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$)
- Learning rates of $\eta = 0.01, 0.001$ work best
- SGD with momentum is also preferred (be it classical or with Nesterov gradient)
- ADAM might just be the best choice overall !
- A recent work suggest that there is a problem with ADAM as $s^{(t+1)}$ could blow up in certain cases and a new algorithm called Yogi* was proposed
- Learning rate scheduling can also be used
 - e.g. $\eta^{t+1} = \alpha \eta^t$ ← learning rate decay

Interactive Demo: Optimizer

In this visualization, you can compare optimizers applied to different cost functions and initialization. For a given cost landscape (1) and initialization (2), you can choose optimizers, their learning rate and decay (3). Then, press the play button to see the optimization process (4). There's no explicit model, but you can assume that finding the cost function's minimum is equivalent to finding the best model for your task.

1. Choose a cost landscape

Select an artificial landscape $\mathcal{J}(w_1, w_2)$.



2. Choose initial parameters

On the cost landscape graph, drag the **red dot** to choose initial parameter values and thus the initial value of the cost.

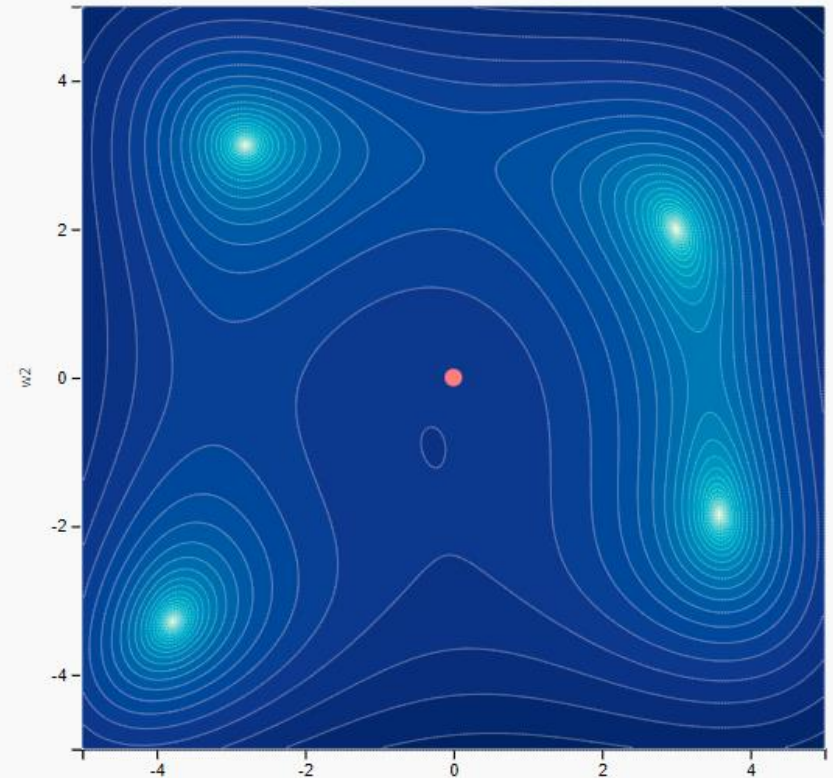
3. Choose an optimizer

Select the optimizer(s) and hyperparameters.

Optimizer	Learning Rate	Learning Rate Decay
☒ Gradient Descent	0.1	0
☒ Momentum	0.1	0
☒ RMSprop	0.1	0
☒ Adam	0.1	0

This 2D plot describes the cost function's value for different values of the two parameters (w_1, w_2). The lighter smaller the cost value.

Himmelblaus Function



Source: Parameter optimization in neural networks: <https://www.deeplearning.ai/ai-notes/optimization/>